Pagerank

CS2HS Workshop

Google

- Google's Pagerank algorithm is a marvel in terms of its effectiveness and simplicity.
- The first company whose initial success was entirely due to "discovery/invention" of a clever algorithm.
- The key idea by Larry Page and Sergey Brin was presented in 1998 at the WWW conference in Brisbane, Queensland.

Outline

- Two parts:

1. Random Surfer Model (RSM) – the conceptual basis of pagerank.

1. Expressing RSM as a problem of eigen-decomposition.

The Key Ideas of Pagerank

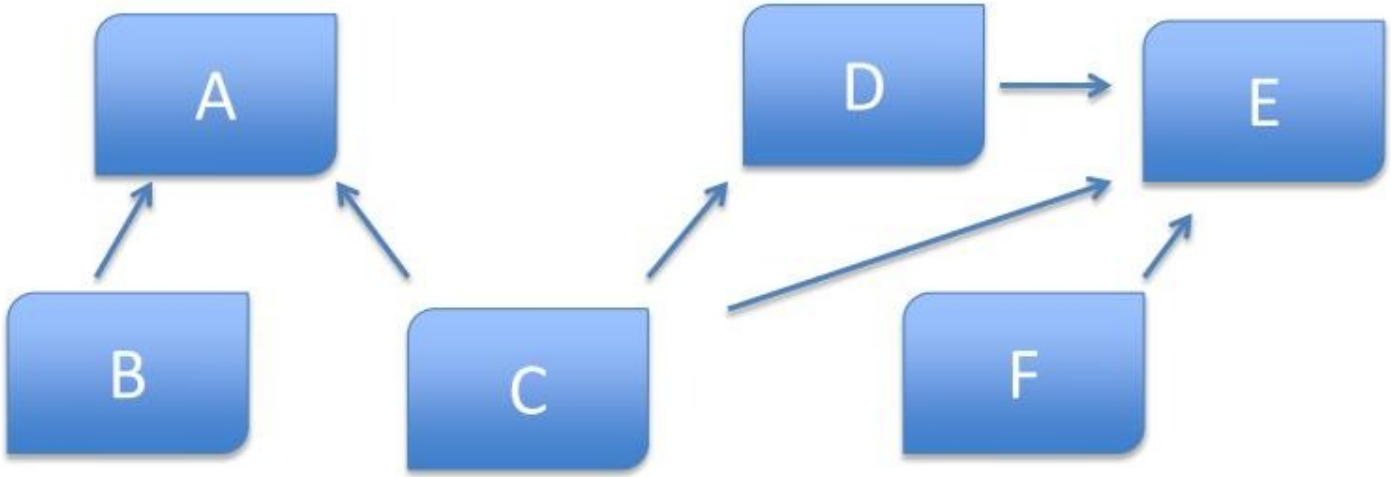
- The Pagerank, at least initially, was based on three key “tricks”

1. The hyperlink trick
2. The authority trick
3. The random-surfer model

Hyperlink trick

- A hyperlink is pointer embedded inside a web page which leads to another page.
- Hyperlink trick: the importance of a page A can be measured by the number of pages pointing to A

Hyperlink example



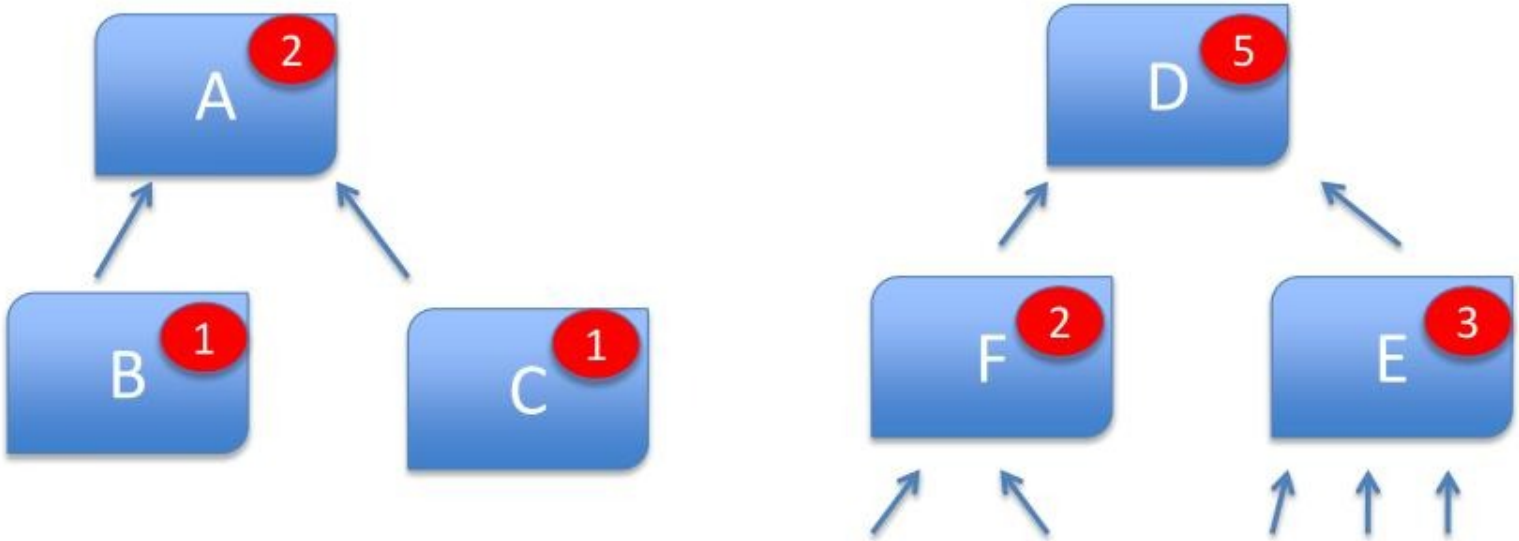
- The importance of A is 2
- The importance of E is 3

- Computers are bad in understanding the content of pages but good at counting
- Importance based just on the count of hyperlinks can be easily exploited

Authority Trick

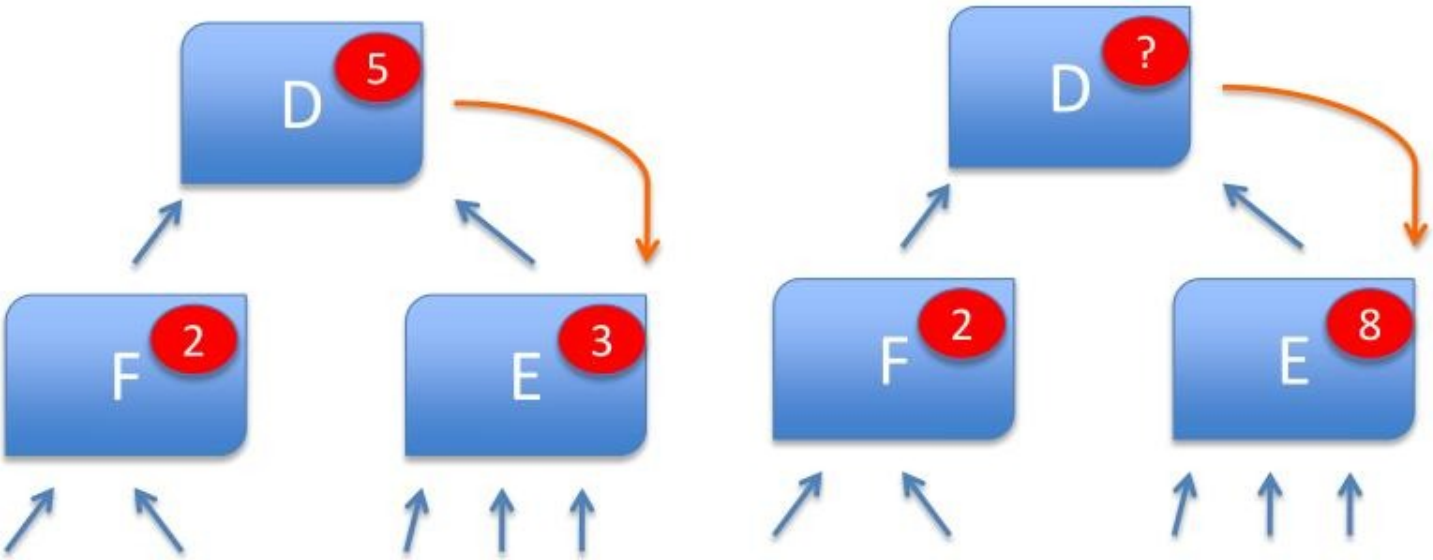
- All links are not equal !

Authority Example



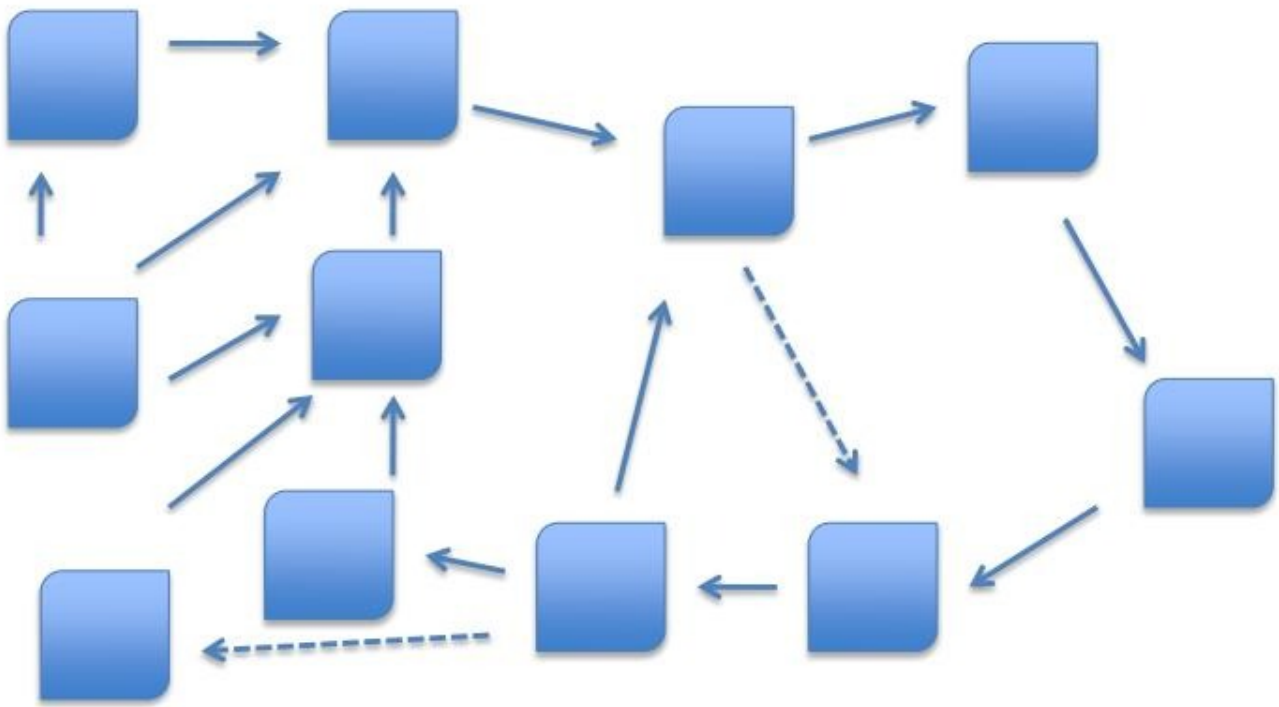
- Authority Count: Cascade the number of counts

Authority Example…cont



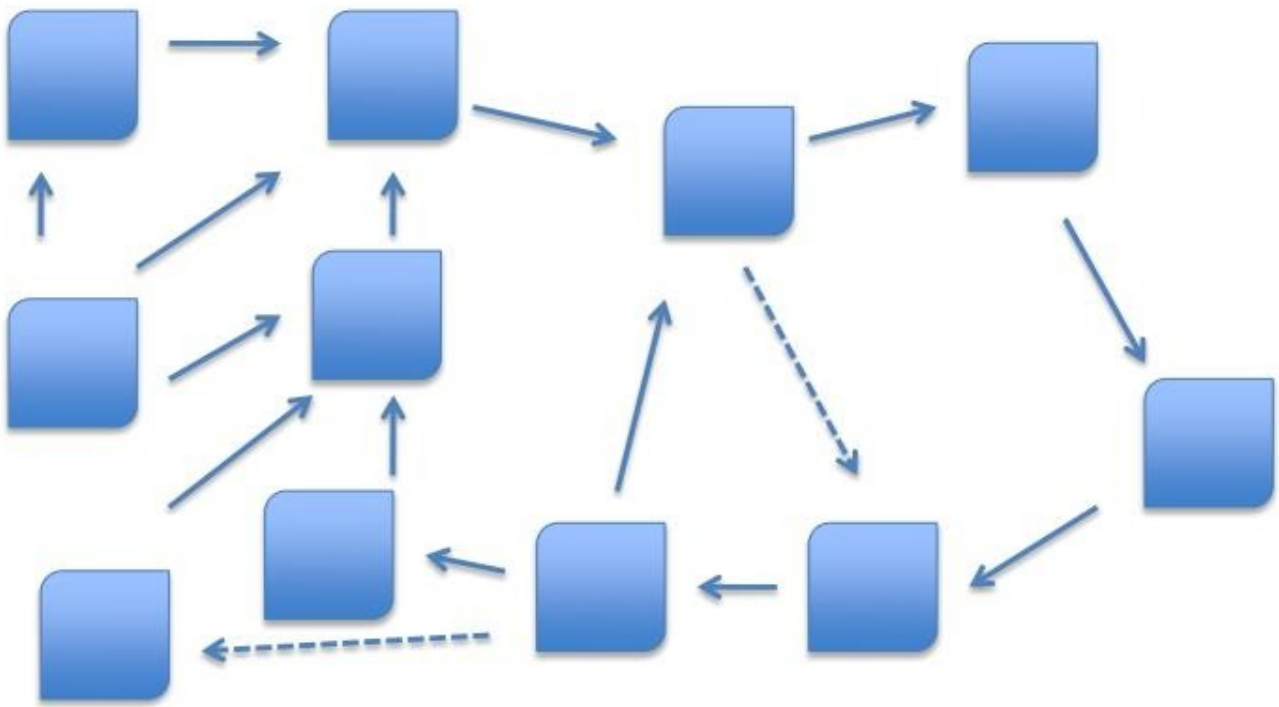
- Presence of cycles will immediately make the authoritative counts redundant !

Random Surfer Model



- A surfer browsing the web by randomly following links, occasionally jumping to a random page

Random Surfer Model



- Combines hyperlink trick, authority trick and solves the cycle problem ! Why ?
- Score or Rank of page A is the proportion of time a random surfer will land up on A

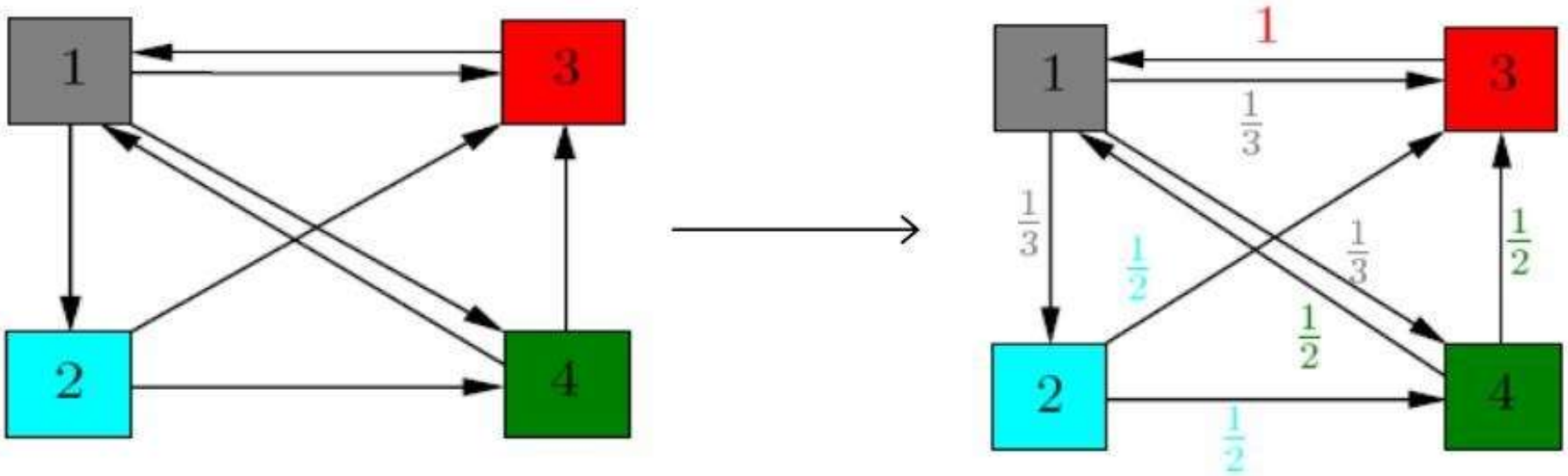
Mathematical Modeling

- Three steps:

1. Model the web as a graph.
2. Convert the graph into a matrix A
3. Compute the eigenvector of A corresponding to eigenvalue 1.

Pagerank: The components of the eigenvector

Example:



Let us denote by A the transition matrix of the graph, $A =$

$$\begin{bmatrix} 0 & 0 & 1 & \frac{1}{2} \\ \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{3} & \frac{1}{2} & 0 & \frac{1}{2} \\ \frac{1}{3} & \frac{1}{2} & 0 & 0 \end{bmatrix}$$

Matrices

- In middle school we learn how to solve simple equations of the form.

$$2x_1 + 4x_2 = 5$$
$$3x_1 + 5x_2 = 6$$

$$\begin{pmatrix} 2 & 4 \\ 3 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 6 \end{pmatrix}$$

Ax = b

- In general, solve equations of the form Ax = b

Special form of Ax=b

- An important special case of $Ax = b$ is the equation of the form

- $Ax = \lambda x$

- λ is called the eigenvalue and the resulting x is called the eigenvector corresponding to λ
- This is one of the most fundamental decomposition in all of mathematics – no kidding!
- Newton, Heisenberg, Schrodinger, climate change, stock market, environmental science, aircraft design,…….

Pagerank

- The pagerank vector is the solution of the equation:
- $Ap = p$ (thus $\lambda = 1$)
- Where A is related to the link matrix
- Note size of A: number or pages on the web –in the billions

Pagerank Equation

- Let p be the page rank vector and L be the link matrix.

$$p = \left(p_1, p_2, \ldots\ldots, p_n\right)$$

$$p_i = (1 - r) + r \sum_{j=1}^{n} \left(\frac{L_{ij}}{c_j} \right) p_j$$

- Here r is the random restart probability (set to 0.15 by Page and Brin)

Pagerank…cont

- Let e by the vector of 1's: e = (1,1,….1)
- Let average pagerank be 1, i.e., $e^t p = n$
- Let $D_c = diag(c)$
- Roll the drums……

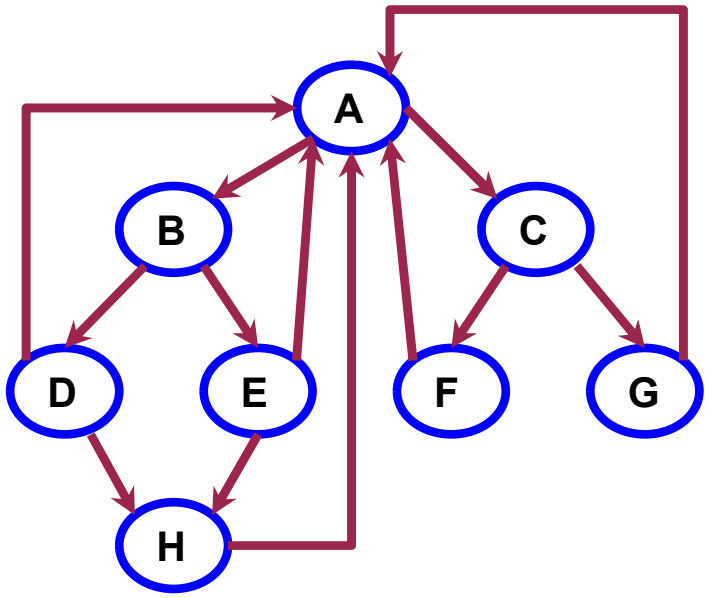
The final page rank equation

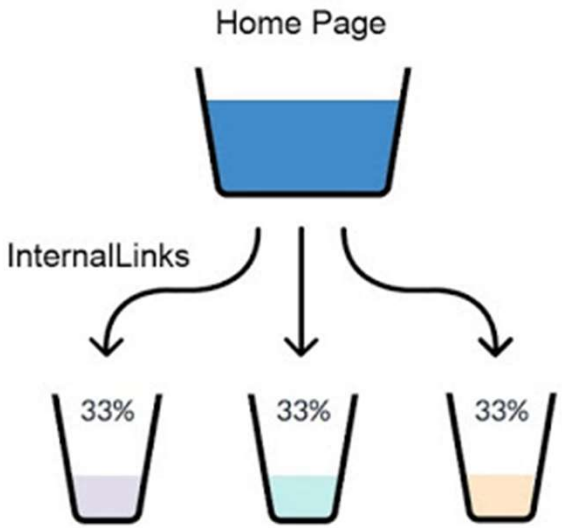
$$p = [(1-r)ee^t / n + rLD_c^{-1}]p = Ap$$

One line code: Open Matlab and type:
[u,v]=eig(A); read of the ranks from the eigenvector corresponding to eigenvalue 1

Lab: Create your web with six pages (with your link structure) and calculate the pagerank. Experiment with different links and confirm if the resulting ranks capture: hyperlink trick, Authority trick and solve the cycle problem

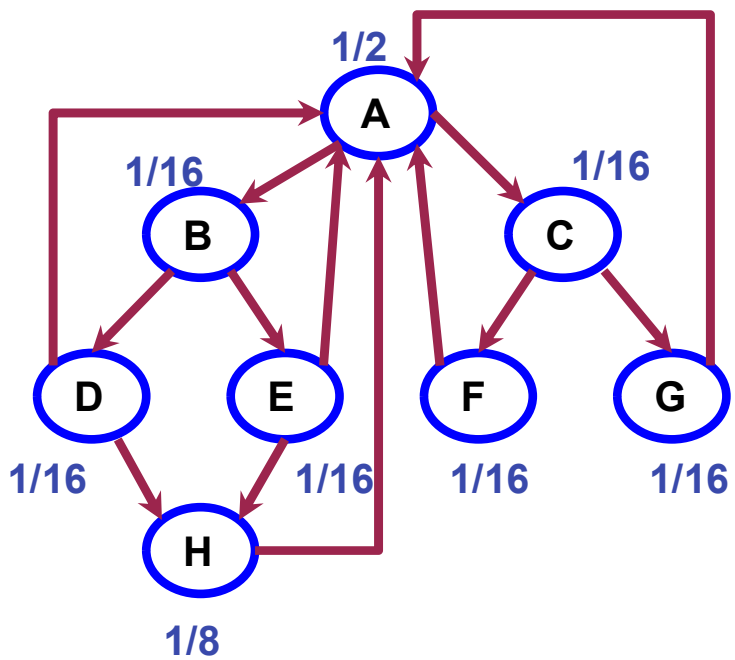
PageRank - Fluid model





Basic PageRank Update Rule: Each page divides its current PageRank ("fluid") equally across its outgoing links and passes these equal shares to the pages it points to. (If a page has no outgoing links, it passes all its current PageRank to itself.) Notice that the total PageRank ("fluid") in the network is unchanged.

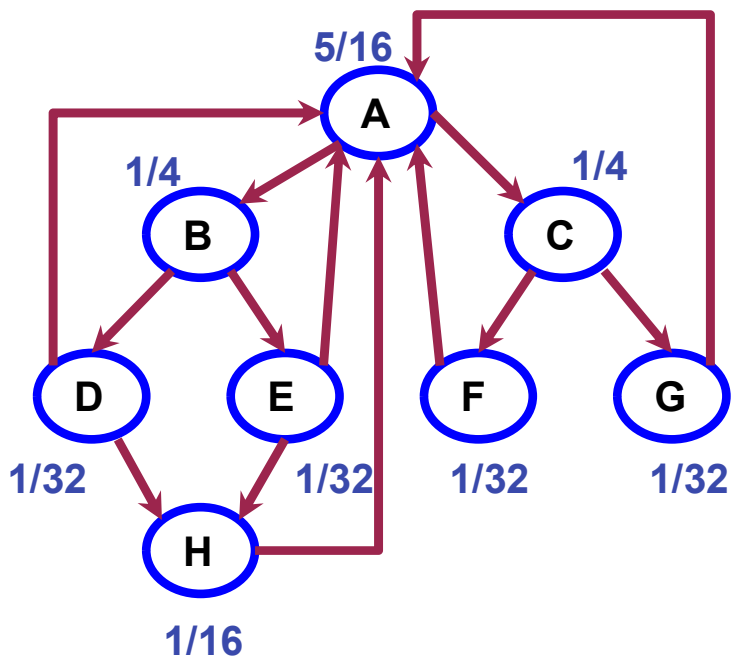
PageRank Algorithm



1. In a network with *n* nodes, we assign all nodes the same initial PageRank, *1/n*
2. We perform a sequence of *k* updates to the PageRank: Each page's PageRank: sum of the shares it receives

Step	A	B	C	D	E	F	G	H
0	1/8	1/8	1/8	1/8	1/8	1/8	1/8	1/8
1	1/2	1/16	1/16	1/16	1/16	1/16	1/16	1/8

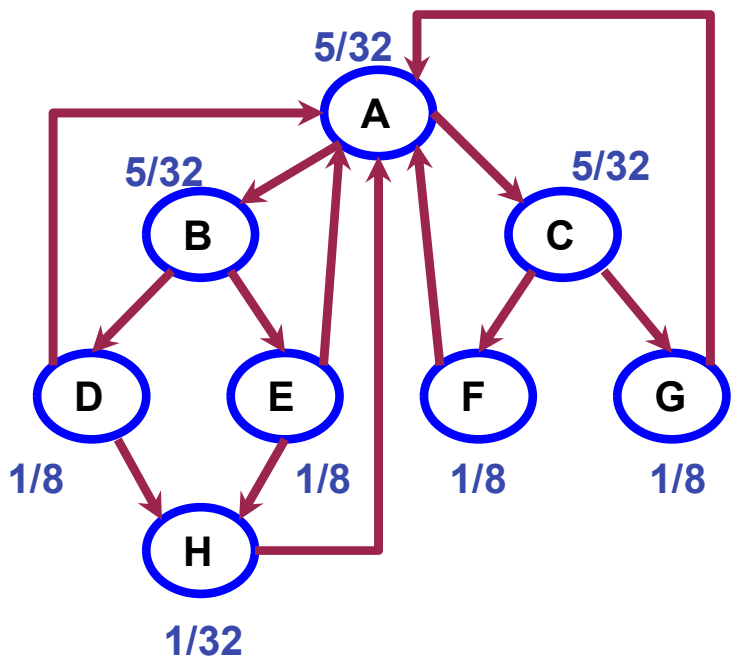
PageRank Algorithm



1. In a network with *n* nodes, we assign all nodes the same initial PageRank, *1/n*
2. We perform a sequence of *k* updates to the PageRank: Each page's PageRank: sum of the shares it receives

Step	A	B	C	D	E	F	G	H
0	1/8	1/8	1/8	1/8	1/8	1/8	1/8	1/8
1	1/2	1/16	1/16	1/16	1/16	1/16	1/16	1/8
2	5/16	1/4	1/4	1/32	1/32	1/32	1/32	1/16

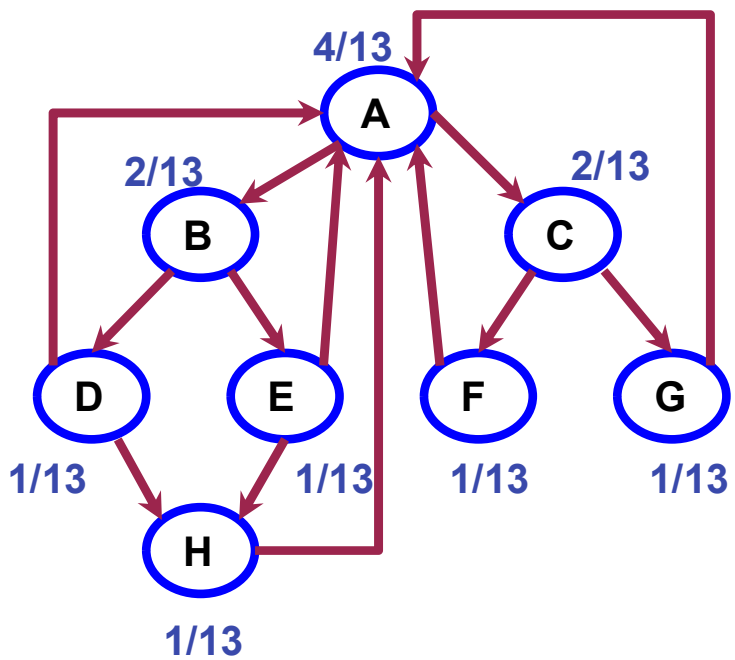
PageRank Algorithm



1. In a network with *n* nodes, we assign all nodes the same initial PageRank, *1/n*
2. We perform a sequence of *k* updates to the PageRank: Each page's PageRank: sum of the shares it receives

Step	A	B	C	D	E	F	G	H
0	1/8	1/8	1/8	1/8	1/8	1/8	1/8	1/8
1	1/2	1/16	1/16	1/16	1/16	1/16	1/16	1/8
2	5/16	1/4	1/4	1/32	1/32	1/32	1/32	1/16
3	5/32	5/32	5/32	1/8	1/8	1/8	1/8	1/32

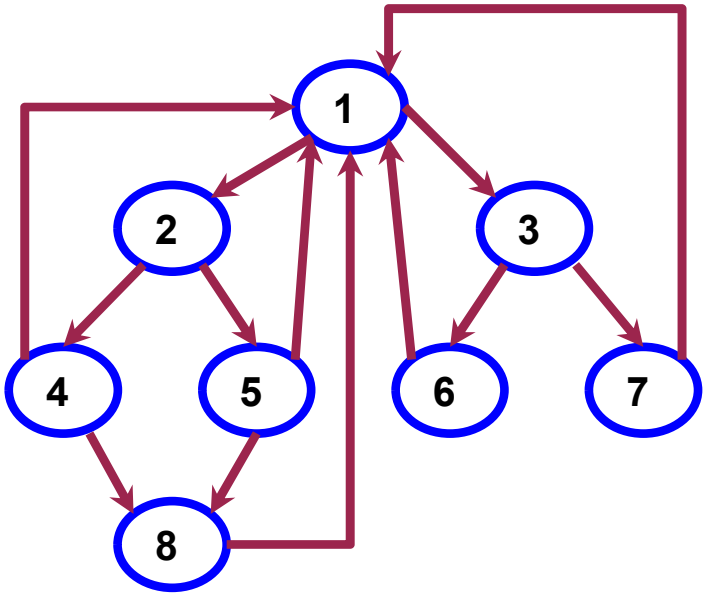
PageRank Algorithm



1. In a network with *n* nodes, we assign all nodes the same initial PageRank, *1/n*
2. We perform a sequence of *k* updates to the PageRank: Each page's PageRank: sum of the shares it receives

Step	A	B	C	D	E	F	G	H
0	1/8	1/8	1/8	1/8	1/8	1/8	1/8	1/8
1	1/2	1/16	1/16	1/16	1/16	1/16	1/16	1/8
2	5/16	1/4	1/4	1/32	1/32	1/32	1/32	1/16
3	5/32	5/32	5/32	1/8	1/8	1/8	1/8	1/32
∞	4/13	2/13	2/13	1/13	1/13	1/13	1/13	1/13

PageRank - Random Walk



$$PR(u) = \sum_{v \in B_u} \frac{PR(v)}{L(v)}.$$

$$(P)_{t+1} = \begin{pmatrix} 0 & 0 & 0 & ½ & ½ & 1 & 1 & 1 \\ ½ & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ ½ & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & ½ & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & ½ & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & ½ & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & ½ & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & ½ & ½ & 0 & 0 & 0 \end{pmatrix} \times (P)_t$$

Assumption: Strongly connected network: each node can reach each other node by a directed path

There is a unique equilibrium probability vector P*, such that: (P*) = (M)(P*)